

九齿 .skill 创新挑战赛道

赛题报告 · T3-1-1

字段	内容
参赛者 / GitHub ID	sunjy2026
赛题编号	T3-1-1
赛季	2026 春季赛
Skill 名称	ninetoothed-operator-skill
提交日期	2026-07-12
硬件	Google Colab T4 GPU
NineToothed 版本	0.26.0

1. 摘要

本报告描述了为 InfiniTensor NineToothed .skill 创新挑战赛 (T3-1-1) 开发的 ninetoothed-operator-skill 作品。该 .skill 是一套可指导 AI 编程助手 (Trae、Cursor、Codex 等) 稳定完成 NineToothed 算子开发、测试、调试、性能分析与 PR 集成任务的工作流说明书。

核心交付：4 类自测算子任务全部通过。Google Colab T4 验证中，correctness tests 28/28 passed, benchmark / generated-source dump / performance tests 16/16 passed, 整体 44/44 validation targets passed across runs; 同时包含多组 benchmark 数据、8 次失败调试迭代记录，以及使用 Skill 的预期改进分析。

2. 赛题理解

本赛道的核心不是让参赛者手写性能最强的算子，而是沉淀一个可复用的 .skill，使统一 AI 智能体在面对陌生隐藏任务时能更稳定地完成开发→测试→调试→性能分析→PR 集成的完整闭环。

评审关注以下五点：

- 任务完成度：AI 看完 skill 后能否从陌生任务推导出正确实现
- 测试验证：correctness test、pytest 输出、参考实现对齐
- 性能意识：benchmark 有具体输入规模、基线、结论
- 仓库合规：只改动 competition skill 目录，命名规范
- 失败诊断：记录失败现象、根因、最小修复、验证闭环

3. .skill 目标与范围

3.1 目标

指导 AI 编程助手完成 NineToothed 算子开发的四类任务：

- 逐元素 / 广播算子 (elementwise)
- 归约 / 分块算子 (reduction)

- 布局敏感算子 (non-contiguous / stride / offset)
- 性能诊断与 benchmark

3.2 不适用范围

- 矩阵乘法 (GEMM) 等复杂 2D 归约算子
- 需要跨行归约的算子 (如 LayerNorm 全局统计)
- NineToothed > 0.26.0 版本的 API 变更
- 多 GPU / 分布式场景

4. 包结构

```
skills/competition/ninetoothed-operator-skill/
├── SKILL.md      # 核心工作流 (189 行), 面向 AI 智能体
├── README.md     # 安装、使用、自测说明
├── HONOR_CODE.md # 诚信声明
├── REFERENCE.md  # 引用与 AI 辅助披露
├── evidence/
│   ├── commands.md
│   ├── environment.txt
│   ├── pytest-correctness.txt
│   ├── pytest-benchmark.txt
│   └── benchmark-summary.md
├── references/
│   ├── dsl-quick-reference.md # NineToothed DSL 速查表
│   ├── repo-structure-index.md # 仓库结构索引
│   ├── operator-patterns.md   # 算子模板
│   ├── v0.26-known-failures.md # NineToothed 0.26.0 常见失败
│   └── benchmarking-and-diagnostics.md # benchmark/AOT/generated-source dump 说明
├── ops/
│   ├── relu.py
│   ├── softmax.py
│   └── rms_norm.py
├── tests/
│   ├── test_relu.py
│   └── test_softmax.py
```

```
|   |— test_rms_norm_noncontiguous.py
|   |— test_softmax_perf.py
|— examples/
|   |— self_test_1_relu.md
|   |— self_test_2_softmax.md
|   |— self_test_3_rms_norm.md
|   |— self_test_4_softmax_perf.md
|— scripts/
|   |— run_benchmark.py
|   |— run_correctness.py
|— reports/
    |— sunjy2026_九齿 skill 创新挑战_T3-1-1_赛题报告.pdf
```

5. 核心 workflow

SKILL.md 定义的 AI 智能体标准执行流程 (Step 0–8, 共 9 步) :

- Step 0 — Read before you write: 先读 README.md、CONTRIBUTING.md、ninetoothed-examples 中的相似算子实现、测试和 benchmark。
- Step 1 — Extract operator contract: 提取输入、输出、shape、dtype、broadcast、边界条件、layout 约束和性能目标。
- Step 2 — Choose arrangement pattern: 根据算子类型选择 elementwise、row-wise reduction、layout fallback 或 benchmark/diagnostics 模式。
- Step 3 — Write application: 只使用 NineToothed/ntl 张量操作; 归约前进行必要 fp32 cast。
- Step 4 — Declare tensors and build kernel: 声明 Tensor rank, 确保 tensors 顺序与 arrangement 签名一致。
- Step 5 — Write correctness tests: 使用 PyTorch reference, 覆盖 shape、dtype、边界条件和非连续输入或 fallback 路径。

- Step 6 — Trigger generated-source dump and triage AOT when required: 自测仅验证 generated-source dump 可触发且 kernel correctness 仍通过；如任务要求源码分析或 AOT，需要额外捕获源码或运行实际 AOT build 命令并保存输出。
- Step 7 — Benchmark: 记录输入规模、PyTorch baseline、GB/s、运行命令和性能结论。
- Step 8 — Diagnose failures: 记录失败现象、诊断路径、根因、最小修复和重跑验证结果。

6. 自测任务 1: ReLU (逐元素)

6.1 任务描述

实现 ReLU 算子: $\text{output} = \max(0, \text{input})$, 覆盖 float32/float16、contiguous/non-contiguous、边界值。

6.2 实现要点

- arrangement: `Tensor(1).tile(ninetoohed.block_size())` — 1D 自动调优
- application: `output[...] = max(0.0, input)` — 匹配 ntops 官方实现风格
- 非连续输入: 使用显式 contiguous-copy fallback; 测试先构造 stride-2 或 transpose 输入, 再与原始非连续 PyTorch reference 对齐验证。

6.3 测试结果

ReLU correctness: 12/12 passed

ReLU benchmark: 2/2 passed

6.4 Benchmark (T4 GPU)

输入规模	耗时	带宽
1M (2 ²⁰) fp16	0.029 ms	145.0 GB/s
16M (2 ²⁴) fp16	0.276 ms	243.4 GB/s

16M 场景下吞吐量更高, 表现接近内存带宽受限模式。

7. 自测任务 2: Softmax (归约)

7.1 任务描述

实现在线两遍 softmax (online two-pass) , 对最后一维归约, 覆盖数值稳定性和非连续输入。

7.2 实现要点

- arrangement: 复用 ntops reduction.py 的 permute + flatten + tile 模式
- application: 两遍循环 — 第一遍更新 max 和 denominator, 第二遍写 output
- 数值稳定性: $\exp(x - \max)$ 防止溢出
- BLOCK_SIZE: 用 _SoftmaxKernel wrapper, 支持显式传参和自动推算两种调用方式

7.3 测试结果

```
python scripts/run_correctness.py --op softmax
9 passed, 34 deselected in 8.00s
```

Softmax benchmark tests are counted under benchmark / performance validation.

7.4 Benchmark (T4 GPU)

Shape	NineToothed	PyTorch
(1024, 512)	136.8 GB/s (0.031ms)	13.8 GB/s (0.303ms)
(1024, 2048)	225.6 GB/s (0.074ms)	208.0 GB/s (0.081ms)
(4096, 4096)	227.8 GB/s (0.589ms)	242.3 GB/s (0.554ms)

该 microbenchmark 中 NineToothed 在三组 shape 上均通过性能验证; 更稳定的小行宽结论以 test_softmax_perf.py 中分离的 NT/PyTorch throughput benchmark 为准。

8. 自测任务 3: RMS Norm (布局敏感)

8.1 任务描述

实现 RMS Norm: $\text{output} = \text{input} / \sqrt{\text{mean}(\text{input}^2) + \text{eps}}$, 重点测试非连续输入 (转置、行切片、列切片) 。

8.2 实现要点

- arrangement: `Tensor(2).tile((1, BLOCK_SIZE))` — 每程序处理一行
- application: `ntl.sum(input2) / input.shape[1] + 1e-6` — 正索引避免 Triton 负索引问题
- eps: Python 字面量 `1e-6`, 避免 `Tensor(0) → pointer<fp32>` 类型错误
- 非连续输入: `.contiguous()` 预处理后传入 kernel

8.3 测试结果

RMSNorm correctness: 7/7 passed
RMSNorm copy-overhead benchmark: 1/1 passed

correctness 测试: `contiguous×3`、`transposed`、`row_slice`、`col_slice`、`rejects_dynamic_eps`。`copy_overhead` 属于 benchmark/performance 测试，单独计入。新增 `test_rms_norm_rejects_dynamic_eps`，验证 `eps != 1e-6` 时会显式抛出 `ValueError`，避免 wrapper 接口语义误导。

8.4 非连续输入 overhead 分析

`contiguous`: 0.041ms | `non-contiguous`: 0.067ms | overhead: 65.6%

`.contiguous()` 复制导致额外开销。该路径是 NineToothed v0.26.0 下的保守 fallback：当原生 stride-aware 2D 归约布局不可安全表达时，优先保证 correctness，并将 copy overhead 作为性能材料记录。由于 Colab T4 benchmark 存在运行间波动，表中数字用于记录自测规模和趋势，不作为固定性能承诺；最终 PR 以 `evidence/pytest-benchmark.txt` 中原始日志为准。

8.5 失败诊断记录 (8 次迭代)

次数	写法	错误	根因
1	eps 作为 <code>Tensor(0)</code>	<code>pointer<fp32></code> 类型错误	<code>Tensor(0)</code> 生成 <code>pointer</code> 类型，不支持算术
2	<code>output[...] = eps</code> 仍 <code>Tensor(0)</code>	同上	同上
3	删除 eps，闭包捕	<code>NameError</code> :	application 不捕获

	获 BLOCK_SIZE	BLOCK_SIZE	外部 Symbol 闭包
4	BLOCK_SIZE 作为默认参数	NameError in generated code	Symbol 在定义时未解析
5	BLOCK_SIZE 作为普通参数	BLOCK_SIZE 未定义	NineToothed 不转发未声明参数
6	除号用 BLOCK_SIZE 闭包	同 NameError	同上
7	input.shape[-1]	Triton 无法求值 (1,BLOCK_SIZE)[-1]	Triton 不支持 constexpr 负索引
8 ✓	input.shape[1]	PASSED	正索引合法, RMSNorm correctness 7/7 通过, copy-overhead benchmark 1/1 通过

9. 自测任务 4：性能诊断与 Benchmark

9.1 任务描述

对 softmax 进行多维度性能分析：跨形状吞吐率对比、generated-source dump trigger、BLOCK_SIZE padding waste 诊断。

9.2 NT vs PyTorch 吞吐率 (test_softmax_perf.py)

计时方法：CUDA Event, warmup=3 次, repeat=50 次, 取均值。带宽公式： $2 \times \text{元素数} \times \text{字节数} / \text{耗时}$ 。

Shape	NineToothed	PyTorch
(512, 256)	17.5 GB/s	42.3 GB/s
(512, 1024)	150.6 GB/s	205.1 GB/s
(512, 4096)	227.8 GB/s	134.2 GB/s
(2048, 4096)	235.5 GB/s	145.5 GB/s

在本次 Colab T4 日志中，较大 row width 场景下 NineToothed 表现更有竞争力；由于 Colab T4 benchmark 存在运行间波动，表中数字用于记录自测规模和趋势，不作为固定性能承诺；最终以 evidence/pytest-benchmark.txt 原始日志为准。

9.3 BLOCK_SIZE padding waste 分析

```
ncols=100: BLOCK_SIZE=128: 0.028ms | BLOCK_SIZE=1024: 0.026ms | speedup: 0.91x
```

对于 ncols=100 的小矩阵，BLOCK_SIZE=128（最优）与 BLOCK_SIZE=1024（10× 过填充）性能接近（0.91×）。T4 内存子系统在小 shape 下能掩盖部分 padding 开销，但过填充仍带来可测量差异。规则仍适用：使用 next_power_of_2(ncols)，对较大 ncols 可避免显著的带宽浪费。

9.4 Generated-source dump trigger

通过 NINETOOTHED_DUMP_GENERATED_SOURCE=1 环境变量，验证 NineToothed 能成功触发 generated-source dump。test_generated_source_dump_trigger 通过 (shape=(8,256), BLOCK_SIZE=256)，并在 dump 触发后验证 kernel correctness；本自测未解析 load/store/mask。

9.5 测试结果

```
Softmax performance / diagnosis tests: 10/10 passed
```

10. 完整测试汇总

```
python scripts/run_correctness.py
28 passed, 16 deselected

python -m pytest tests/ -m benchmark -v -s --tb=short
16 passed, 28 deselected
```

Overall: 44/44 selected validation targets passed across correctness and benchmark runs.

测试文件	测试分布	场景
test_relu.py	12 correctness + 2 benchmark	逐元素、非连续、benchmark

test_softmax.py	9 correctness + 3 benchmark	归约、数值稳定性、非连续、benchmark
test_rms_norm_noncontiguous.py	7 correctness + 1 copy-overhead benchmark	布局敏感、contiguous/非连续 fallback 对比
test_softmax_perf.py	10 benchmark / diagnosis	性能诊断、generated-source dump trigger、padding waste

11. 定性改进分析

本节不是严格 no-skill vs with-skill 对照实验，而是基于自测调试过程中观察到的高频失败模式，总结 skill 预期降低的错误类型。严格对照实验需要在相同模型、相同时间预算、相同工具权限和相同初始仓库下分别运行，并保存 prompt、输出、diff 和 pytest 结果；本提交暂未包含该实验。

常见失败模式	无 Skill 时的预期风险	使用 Skill 后的预期改进
Tensor(0) scalar 用法	随机尝试，反复遇到 pointer<fp32> 错误	跳过，直接用 Python 字面量
BLOCK_SIZE 传递	尝试闭包/默认参数，多次 NameError	直接使用 Symbol + 调用时显式传参
非连续输入处理	不清楚何时需要原生 stride 支持或 .contiguous() fallback，测试随机失败	明确区分原生 stride 支持与 contiguous-copy fallback，并记录 correctness 与 overhead
Triton 负索引问题	不知道 constexpr tuple 不支持负索引	知道用 shape[1] 代替 shape[-1]
benchmark 模式	无固定模式，结果不可复现	固定 CUDA Event 计时 + GB/s 计算公式
失败诊断	随机修改，无系统方法	按 SKILL.md Step 8 流程：现象→根因→最小修复→验证

12. 安全与合规

- 无 API key、账号凭据、联网依赖
- 无硬编码隐藏评测答案
- 无删除或绕过测试的逻辑
- 仅修改 skills/competition/ninetoothed-operator-skill/ 目录
- 安装依赖后，所有自测任务可在 Google Colab T4 GPU 环境中复现；运行过程不依赖 API key、账号凭据或隐藏评测答案
- AI 辅助范围详见 REFERENCE.md

13. 局限性与后续维护

已知局限

- NineToothed v0.26.0 下，部分 stride-aware 2D 归约布局不可安全表达；本 skill 指导 AI 优先识别 layout 约束，必要时使用 `.contiguous()` fallback，并记录 copy overhead；Colab T4 多次运行中 overhead 有波动（约 65%–90%）。
- `Tensor(0) scalar` 参数生成 `pointer<fp32>`，限制了可传标量参数的场景
- `eps` 固定为 `1e-6`；wrapper 会拒绝其他 `eps` 值并抛出 `ValueError`，避免用户传入 `eps=1e-5` 但实际仍按 `1e-6` 计算。
- benchmark 数据来自 T4，A100/H100 上结论可能不同

后续可维护方向

- 升级到支持 stride-aware 布局的 NineToothed 版本后，移除 `.contiguous()` 预处理
- 增加 mm（矩阵乘法）算子自测任务
- 补充 fp16/bf16 dtype 的 benchmark 对比